

SEQUENTIAL CIRCUITS

- In the first part of the course, **combinational circuits** are covered.
The outputs of the combinational circuits depend only on the **current** inputs.
- In **sequential circuits**, the outputs depend both on the inputs and the "state" of the circuit.
Memory units are required to store (remember) the state of the circuit.
- For example, vending machines keep track of (remember) the coins that were inserted into the machine. With each coin, the state of the machine (total amount of inserted coins) is updated.
- There are two types of sequential circuits:
 - **Synchronous sequential circuits:** Their state can change at a discrete instance of time.
All memory elements are synchronized by a common clock signal. Therefore these circuits are also called "clocked synchronous sequential" circuit.
 - **Asynchronous sequential circuit:** Their state can change at any instant of time depending upon the input signals.
 - In this course we will deal only with clocked synchronous sequential circuits, because nearly all sequential logic today is clocked synchronous.
 - For example microprocessors are clocked synchronous sequential circuits.

Finite State Machine (FSM) Model

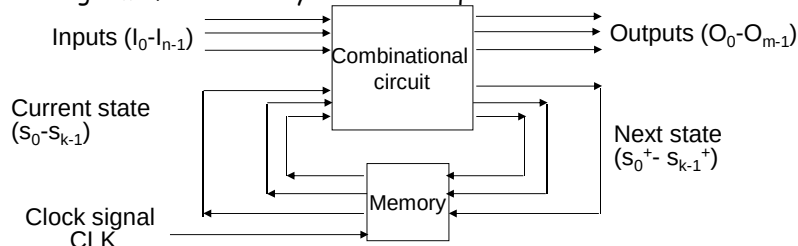
Sequential circuits are designed using "finite state machine - FSM" model.
This model is also used in the design of many other systems.

- When the machine is started, it is in a certain state (initial state: s_0).
- An output is produced depending on the inputs and the current state.
- Transition into a new state happens depending on the input and the current state.

A FSM has two parts:

- Combinational circuit for logical operations
- Memory unit to remember the current state

Block diagram of a clocked synchronous sequential circuit:

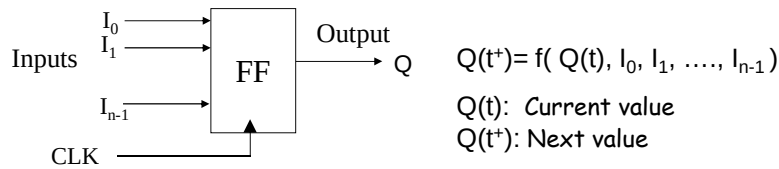


We will see details of the FSM and clocked synchronous sequential circuits (in chapters 7 and 8) after we cover memory units.

Memory Units

'Flip-flop': One-bit memory unit.

They are designed as logical circuits with multiple inputs and a single output.



Q output shows the current value of the flip-flop (0,1). This value is the state of the flip-flop.

The next value of the output Q (denoted by $Q(t^+)$ or Q^+) is a function of the current state (denoted by $Q(t)$ or Q) and the current inputs.

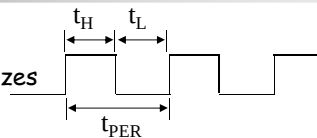
Clock signal (denoted as CLK) determines the time when the next state function is evaluated and the output of the flip-flop changes its value.

The output of the flip-flop can only change when the clock signal is active (the definition of being active will be described in the next slides).

If the clock signal is not active, flip-flop output will not change even if the input values change.

Clock Signal:

Clock signal is a periodic square wave that synchronizes the gates in the circuit.



Logic units with clock signal input (such as flip-flops) are enabled only when the clock signal is active. If clock signal is not active, they preserve their state.

There are two types of units in terms of clock signal activation:

- a) Level-triggered units
- b) Edge-triggered units

Level-triggered units use a level of the clock signal (1 in positive logic) as active.

These units activate and change their outputs when the clock signal is at **high** level.

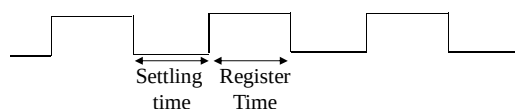
They preserve their state when the clock signal is at **low** level.

When the clock signal is at high level ("1"), the inputs should not change as they are processed.

Otherwise the output of the sequential circuit is undetermined (random).

This time is called **register time**.

The inputs can change when the clock signal is 0. This time is called **settling time**.

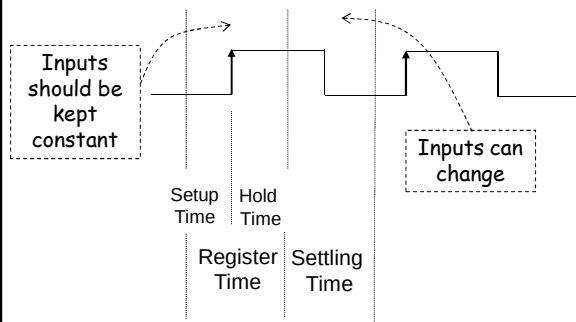


Edge-triggered units:

These units use an edge (rising edge in positive logic) of the clock signal as active. Positive edge-triggered units use 0→1 transition of the clock signal (rising edge) to change their state and output. At other times, they preserve their state.

As the inputs are used (processed) during 0→1 transition, inputs should be kept constant for certain time before and after the transition.

Otherwise, the output of the sequential unit is undetermined (random).



The **register time** is the sum of the setup and hold times.

Setup time is the minimum amount of time the data signal should be held steady **before** the clock transition.

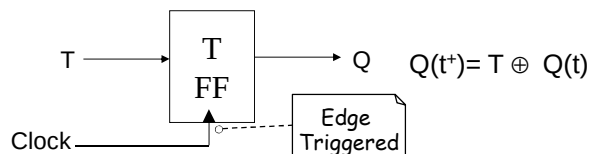
Hold time is the minimum amount of time the data signal should be held steady **after** the clock transition.

The inputs should be kept constant during the register time so that the sequential circuit works correctly.

In **negative logic**, all transactions happen at 1→0 transition (falling edge).

Example: Edge-triggered Toggle (T) Flip-Flop

Before detailed explanation of flip-flops, a T flip-flop is used as an example.



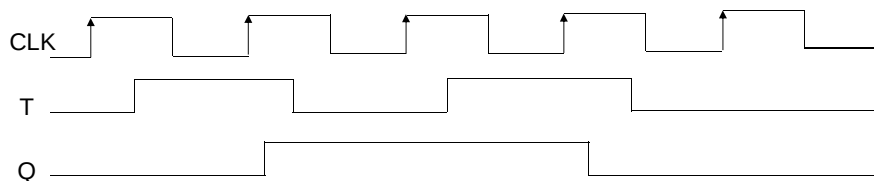
The next output (state) of the T flip-flop $Q(t^+)$ is equal to its current output (state) XOR its input (T).

According to this, the output of the flip-flop does not change when $T=0$ because:

$$0 \oplus x = x$$

The output of the flip-flop is complemented when $T=1$ because:

$$1 \oplus x = x'$$



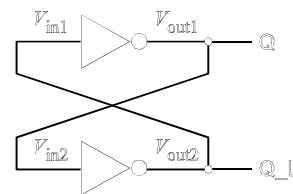
Bistable (Two stable states) Circuit

A bistable circuit can be implemented with two inverters.

The inputs of the gates are connected to the outputs (feedback) of each other.

This bistable circuit has no external inputs.

This circuit will always be in one of the **two possible stable states**.



1. If the output of the inverter at the top is $V_{out1} (Q) = 0$, the input of the inverter at the bottom becomes $V_{in2} = 0$, and its output becomes $V_{out2} (Q_L) = 1$. This is a stable state as this state requires $Q = 0$ and $V_{in1} = 1$.

State 1: $V_{in1} = 1, V_{out1} = V_{in2} = 0, V_{out2} = 1$

2. If the output of the inverter at the top is $V_{out1} (Q) = 1$, the input of the inverter at the bottom becomes $V_{in2} = 1$, and its output becomes $(Q_L) = 0$. This is also a stable state as it requires $V_{in1} = 0$ and $Q = 1$.

State 2: $V_{in1} = 0, V_{out1} = V_{in2} = 1, V_{out2} = 0$

This circuit has two stable states: $Q=0$ and $Q=1$

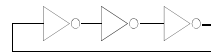
It is impossible to control (change) the state of the circuit as it has no inputs.

When this circuit is turned on, it takes a random state.

Examples of **unstable** circuits:

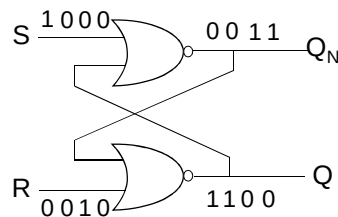


or

**S-R (Set-Reset) Latch**

S-R Latch is a one-bit memory built with two NAND or two NOR gates.

All other latches and flip-flops can be built upon this fundamental memory unit with certain extensions.

S-R latch with NOR gates:

S: Set
 R: Reset
 Q: Output (State)
 Q_N: Complemented output (Q')

Recall: When an input of a NOR gate is "1", the output will be "0" regardless of the other input.

S	R	Q	Q _N	
1	0	1	0	
0	0	1	0	After S=1, R=0
0	1	0	1	
0	0	0	1	After S=0, R=1
1	1	0	0	Forbidden input

- S input is used to write (store) "1" to the latch, R input is used to write "0".
- If both inputs are "0", SR latch preserves its state.
- Both inputs should not be "1" at the same time.

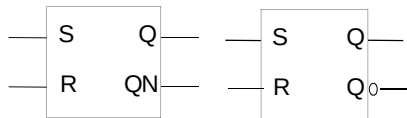
Next value of the output (state) $Q(t+1)$ depends on the inputs and current output (state) $Q(t)$.

The truth table and the logical expression of the SR latch can be expressed as follows:

$Q(t)$	S	R	$Q(t+1)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	forbidden(Φ)
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	forbidden(Φ)

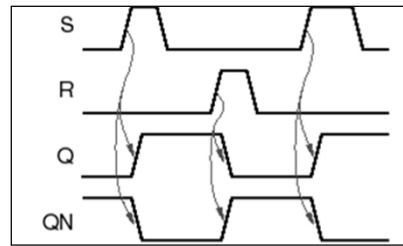
$Q(t+1)$	$Q(t)$				S	
	00	01	11	10		
0			Φ	1		
1	1		Φ	1		
					R	

$$Q(t+1) = S + Q(t)R', \quad SR=0$$



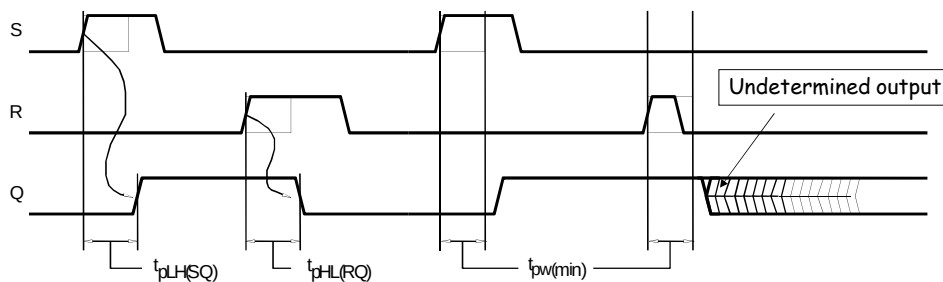
Latches are not triggered by a clock signal.

Flip-flops are clock-triggered memory units.



Because of the propagation delay, the changes in the S and R inputs will not affect the output simultaneously.

During the delay, the inputs should be kept constant. Otherwise, the value of the output is undetermined (random).



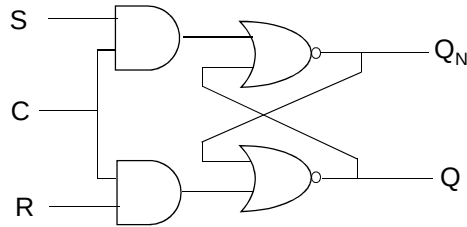
$t_{PLH(SQ)}$: Delay in the 0-1 transition of the output when S changes.

$t_{PHL(RQ)}$: Delay in the 1-0 transition of the output when R changes.

$t_{pw(min)}$: Minimum duration when the inputs should be kept constant.

S-R Latch with Enable Input

AND gates are connected to S and R inputs so that the latch is active only when it is enabled.



S: Set

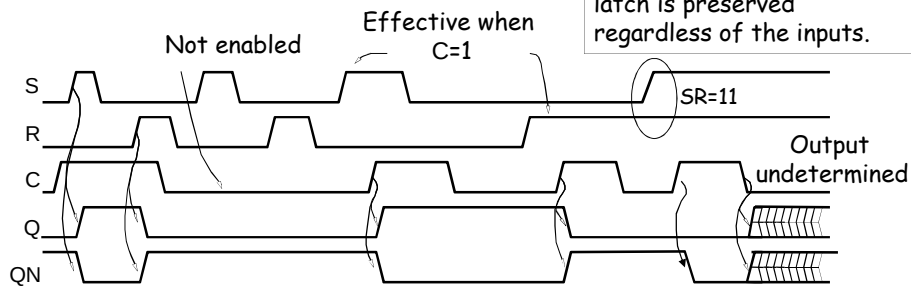
R: Reset

Q: Output (State)

 Q_N : Complemented output (Q')

C: Enable input

The value of the latch can only be changed when $C=1$. When $C=0$, the value of the latch is preserved regardless of the inputs.

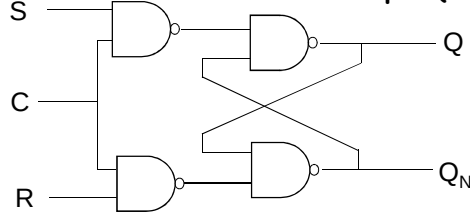


<http://www.faculty.itu.edu.tr/buzluca>
<http://www.buzluca.info>

Translated by Dr. Mustafa KAMAŞAK

© 2011-2014 Dr. Feza BUZLUCA

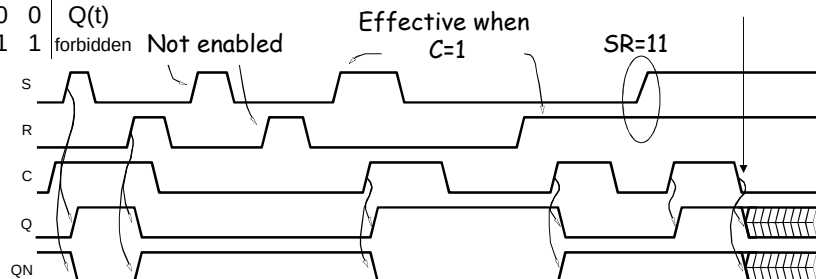
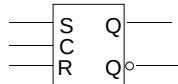
6.11

S-R Latch with Enable Input (with only NAND Gates)

S-R Latch implemented with NOR and AND gates (slide 6.11) can also be implemented using only NAND gates.

If the forbidden input ($SR=11$) is applied to the latch, both Q and Q' outputs will be 1. If the latch is disabled in this state, the value in the latch is undetermined.

C	S	R	$Q(t+1)$
0	X	X	$Q(t)$
1	1	0	1
1	0	1	0
1	0	0	$Q(t)$
1	1	1	forbidden



<http://www.faculty.itu.edu.tr/buzluca>
<http://www.buzluca.info>

© 2011-2014 Dr. Feza BUZLUCA

6.12

Difference between Latch and Flip-flop:

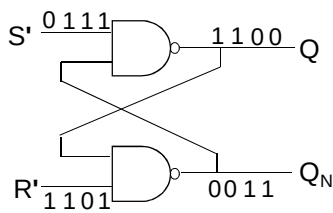
Latches are not triggered by clock signal. The value of the latch can be changed whenever it is enabled.

Memory units that are triggered by clock signal are called **flip-flop**.

S-R Latch with NAND Gates (S'-R' Latch)

S-R latches can be implemented using NAND gates instead of NOR.

These units are called **S'-R'** latch.



S': Complement of Set
R': Complement of Reset
Q: Output (State)
Q_N: Complemented output (Q')

This latch is different than the one in slide 6.12.

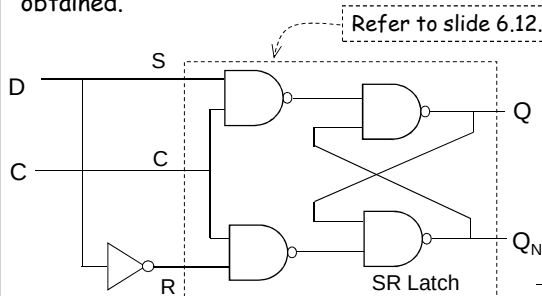
Recall: If an input of a NAND gate is "0", the output will be "1" regardless of the other input.

S'	R'	Q	Q _N	
0	1	1	0	
1	1	1	0	After S'=0, R'=1
1	0	0	1	
1	1	0	1	After S'=1, R'=0
0	0	1	1	Forbidden inputs

Delay (D) Latch

Other types of latches can be built upon S-R latch with some additions.

If the S and R inputs of the SR latch are connected with an inverter, D-latch is obtained.



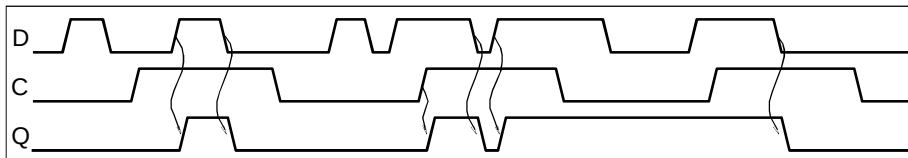
C=1	D	S	R	Q(t ⁺)
0	0	1	0	0 (Reset)
1	1	0	1	1 (Set)

If C=1, the value of D input is written to the latch.

If C=0, latch preserves its previous value.

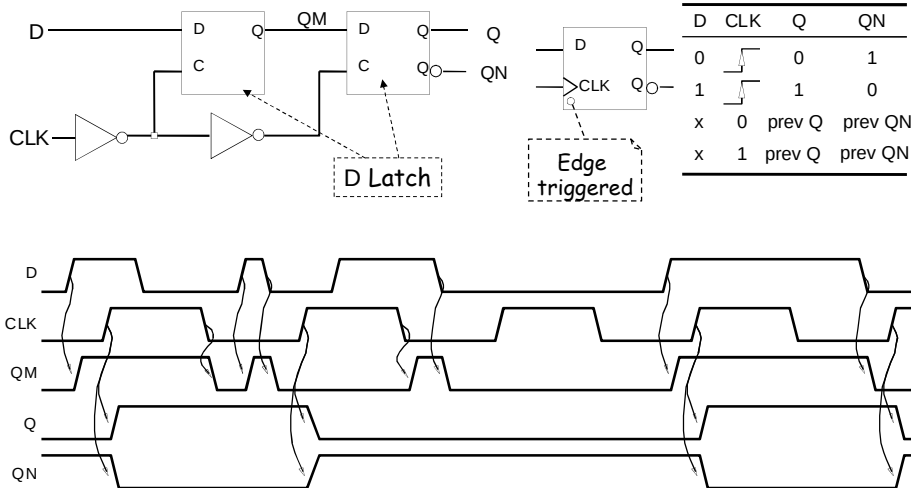
C	D	Q(t ⁺)	Q _N (t ⁺)
1	0	0	1
1	1	1	0
0	X	Q(t)	Q _N (t)

$$Q(t+1)=D$$



Rising Edge Triggered D Flip-flop

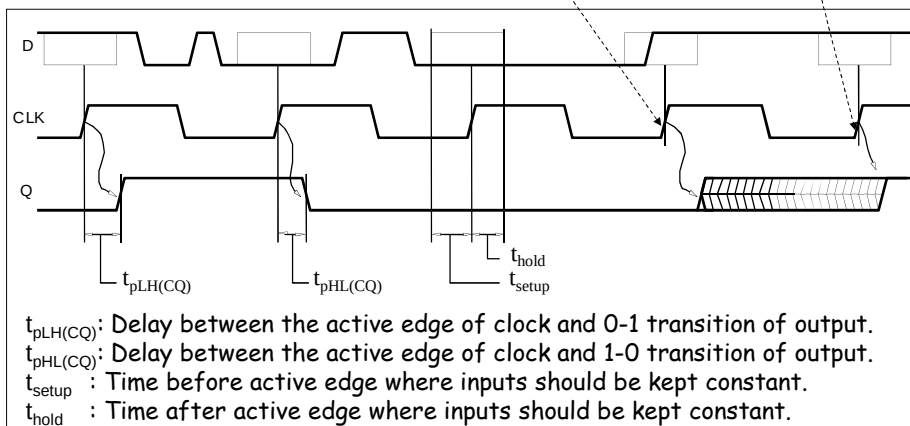
Latches change their value (depending on their input) when they are enabled.
Flip-flops on the other hand, change their value when the clock signal is active.



Timing properties of the rising edge triggered D flip-flop

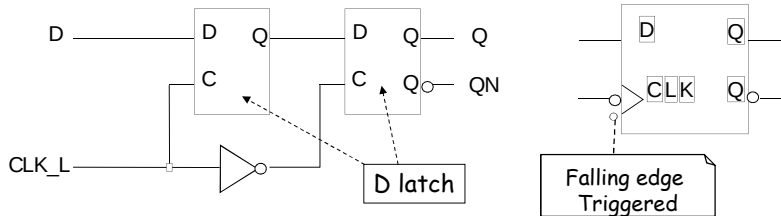
Output is undetermined as the inputs are not kept constant during the setup time.

Output gets a valid value (1 in this example).



Falling Edge Triggered D Flip-flop

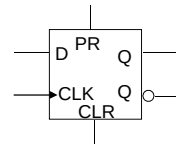
Data is written to D flip-flop during the falling edge of the clock signal.



D	CLK_L	Q	QN
0	0	0	1
1	1	1	0
x	0	prev Q	prev QN
x	1	prev Q	prev QN

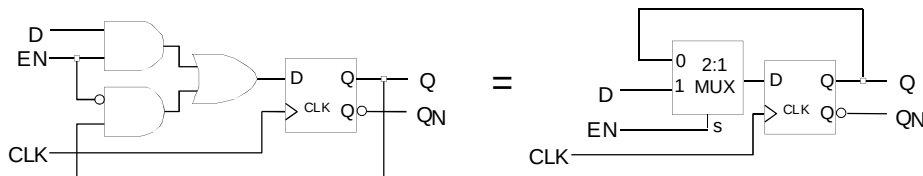
In flip-flops (especially to write an initial value), there may be asynchronous (independent of clock signal) inputs. To write "1" to the flip-flop PR (*Preset*), to write "0" CLR (*Clear*) inputs can be used.

Asynchronous inputs change the value of flip-flop even if the clock signal is not active.

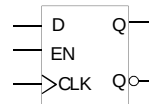


Edge Triggered D Flip-flop with Enable Input

Flip-flops may also have an enable input. If the enable input is active, the value of the flip-flop can be changed. Otherwise, it preserves its value regardless of its inputs.



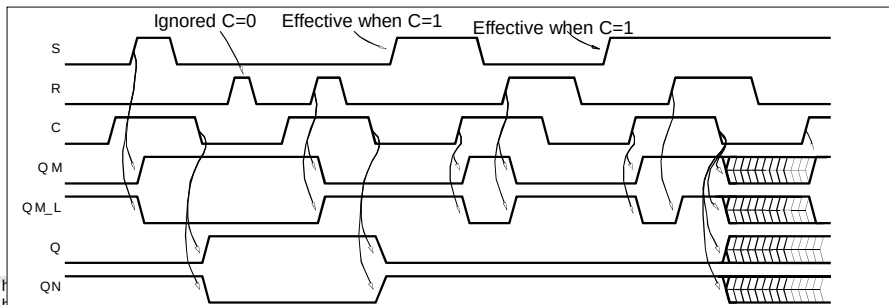
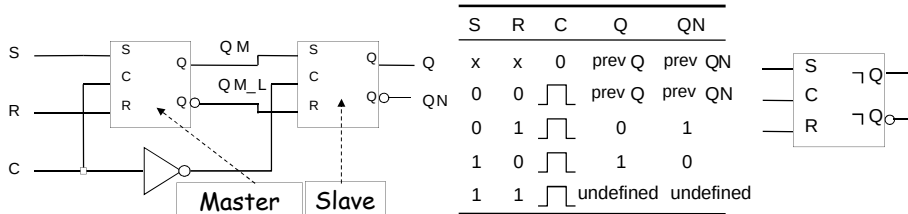
D	EN	CLK	Q	QN
0	1	0	0	1
1	1	1	1	0
x	0	0	prev Q	prev QN
x	x	0	prev Q	prev QN
x	x	1	prev Q	prev QN



Master/Slave SR Flip-flop

Master/slave flip-flops are pulse triggered units. The value (output) of this type of flip-flops changes only at the falling edge of the clock signal.

However, next value of the flip-flop can be determined while the clock signal is 1.

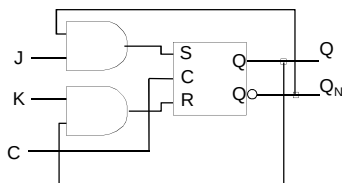


6.19

JK Latch

The problem of forbidden inputs of the SR latches ($S=1, R=1$) is solved in JK latches. These units function similar to SR latches. Input J sets and input K resets the output.

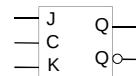
When both inputs are "1" ($J=1, K=1$), the value (output) of the latch is complemented.



J	K	C	Q	Q _N
x	x	0	prev Q	prev Q _N
0	0	1	prev Q	prev Q _N
0	1	1	0	1
1	0	1	1	0
1	1	1	prev Q _N	prev Q

C=1	J	K	Q(t)	S	R	Q(t+1)	Operation
0	0	0	0	0	0	0	Don't change
0	0	1	0	0	0	1	
0	1	0	0	0	0	0	Reset
0	1	1	0	0	1	0	
1	0	0	1	0	0	1	Set
1	0	1	1	0	0	1	
1	1	0	1	0	0	1	Complement
1	1	1	1	0	1	0	

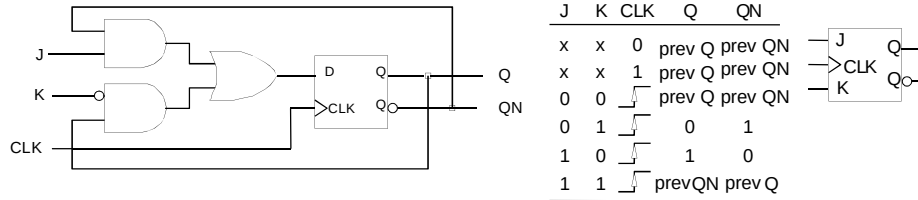
$$Q(t+1) = J \cdot Q(t)' + K' \cdot Q(t)$$



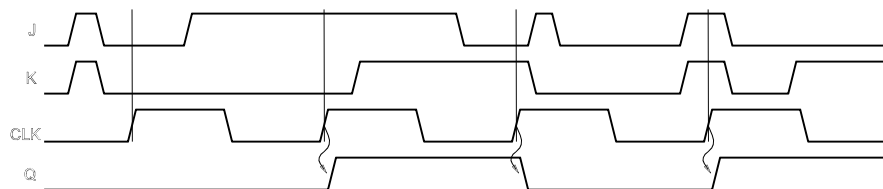
Edge Triggered JK Flip-flop

An edge-triggered JK flip-flop can be implemented using an edge triggered D flip-flop and logical gates.

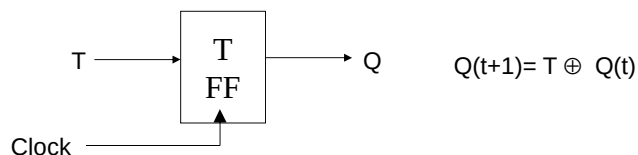
In this unit, the J and K inputs are processed only at the active edge of the clock signal.



$$Q(t+1) = J \cdot Q(t)' + K' \cdot Q(t)$$



Edge Triggered Toggle (T) Flip-flop



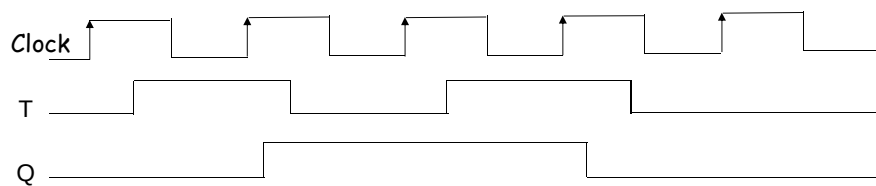
The output (value) of a T flip-flop is determined by XOR operation of the input (T) and the current value.

Therefore, if the input is 0 ($T=0$), the value of the flip-flop is preserved as:

$$0 \oplus x = x$$

If the input is 1 ($T=1$), the value of the flip-flop is complemented (toggled) as:

$$1 \oplus x = x'$$



Characteristic Equations of the latches and flip-flops:

The functional behavior of a latch or flip-flop can be described by a **characteristic equation** that specifies the flip-flop's (or latch's) next state as a function of its inputs and current.

Characteristic equations of flip-flops:

$$\text{SR FF: } Q(t+1) = S + R' \cdot Q(t), \quad SR=0$$

$$\text{JK FF: } Q(t+1) = J \cdot Q(t)' + K' \cdot Q(t)$$

$$\text{D FF: } Q(t+1) = D$$

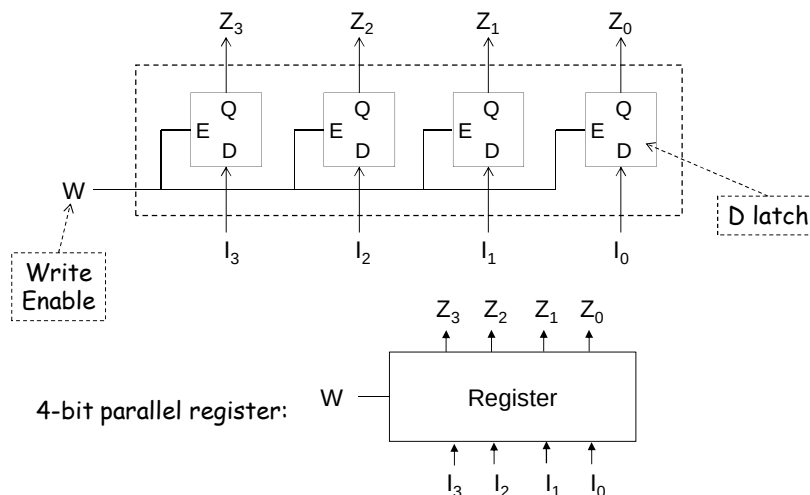
$$\text{T FF: } Q(t+1) = T \oplus Q(t)$$

Registers

Registers are memory units that can hold n-bit binary numbers.

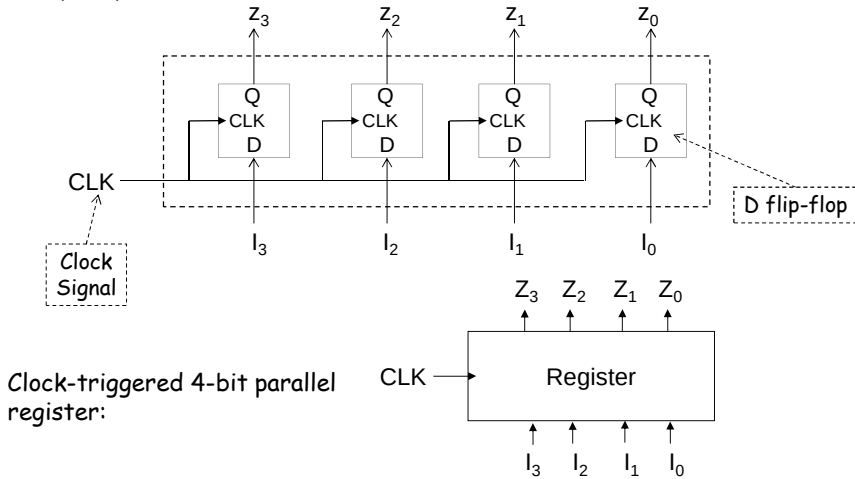
Example: 4-bit parallel register

D latches are used.



Example: 4-bit parallel register with clock signal

D flip-flops are used.



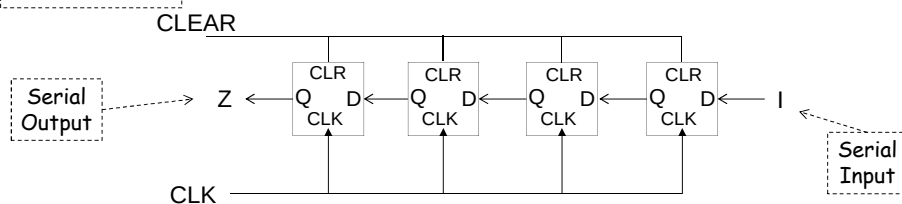
Note: If D flip-flops with enable (E) inputs are used then the register will contain both the clock signal (CLK) and write-enable (W) input.

Serial Shift Registers

Serial shift registers can hold and shift binary numbers to left or right one bit at each active transition of the clock signal.

Example: 4-bit serial left shift register

To reset all bits

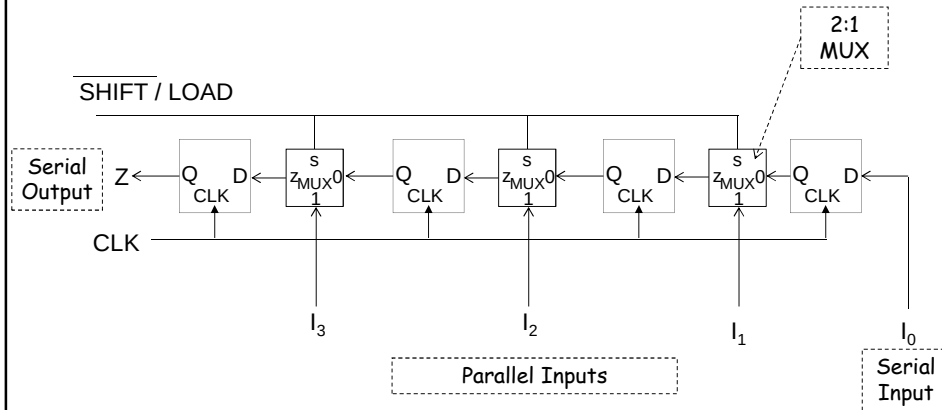


By changing the order of the flip-flops a right-shift register can be implemented.

In this design it is not possible to load an initial value to the register (except zero) in parallel.

Example: 4-bit serial left shift register with parallel load

In this design we can load an initial value to the register in parallel.



SHIFT / LOAD = 0 : shift

SHIFT / LOAD = 1 : load